

Chương 14: Thị giác Máy tính Chuyên sâu sử dụng Mạng Nơ-ron Tích chập (CNN)

Giảng viên: [Tên Giảng Viên]
Đại Học Đông Á
Ngày 11 tháng 8 năm 2026

Gemini Notebook

Nội dung Chương 14



1. Vỏ Não Thị Giác và Nguồn Gốc CNN



2. Các thành phần của CNN (Tích chập & Gộp)



3. Kiến trúc CNN Tiêu Biểu (Lịch sử & Hiện đại)



4. Định Vị Đối Tượng (Object Localization)



5. Phát hiện Đối Tượng (Object Detection)



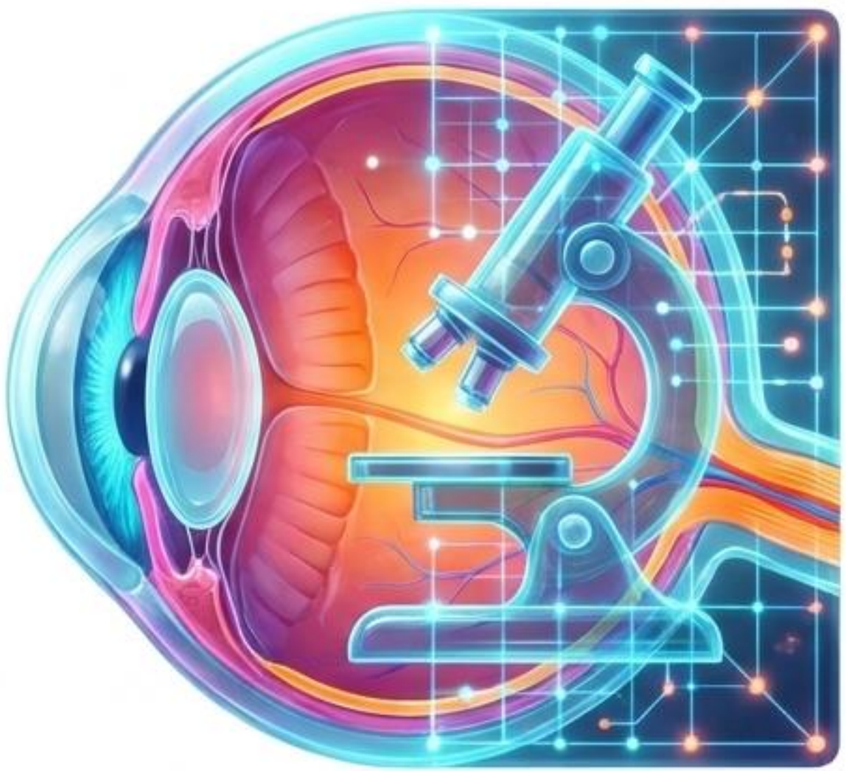
6. Phân đoạn Ngữ nghĩa (Semantic Segmentation)



Tổng Kết

Gemini Notebook

Khởi nguồn của Thị giác Máy tính



Năm 1959, David Hubel và Torsten Wiesel thực hiện một thí nghiệm kinh điển trên mèo, phát hiện ra cấu trúc của vỏ não thị giác (Visual Cortex).

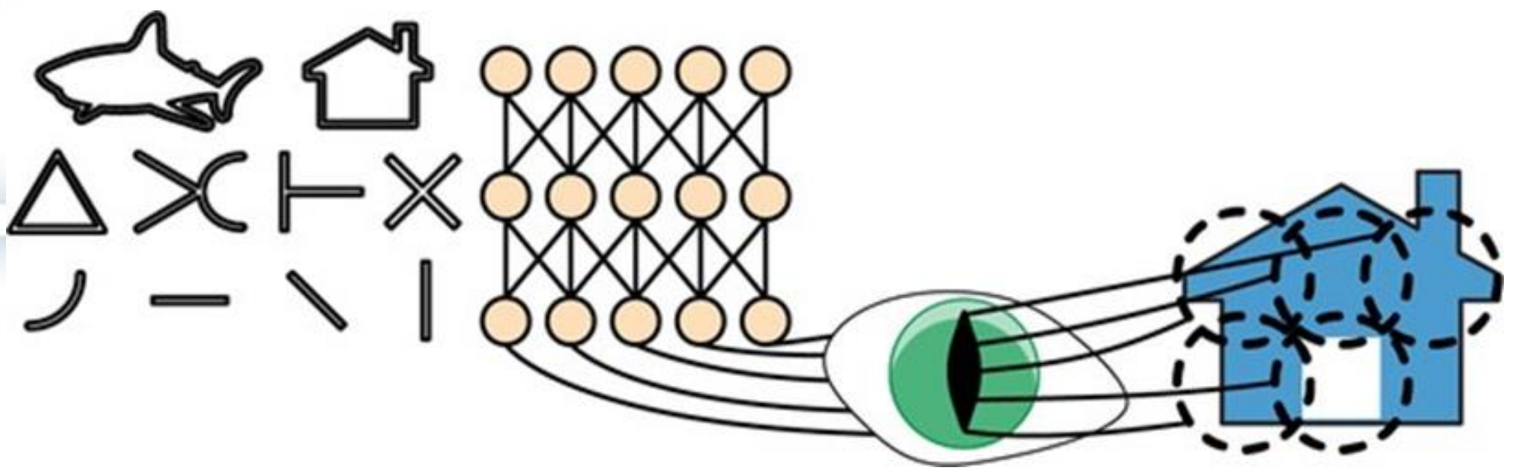
Họ phát hiện rằng các nơ-ron trong vỏ não thị giác có một trường tiếp nhận cục bộ (local receptive field). Một số nơ-ron chỉ phản ứng với các đường ngang, một số nơ-ron khác phản ứng với đường dọc hoặc đường chéo.

Các nơ-ron này được sắp xếp theo cấu trúc phân cấp: Nơ-ron cấp thấp nhận diện các đường nét đơn giản, nơ-ron cấp cao kết hợp chúng để nhận diện hình dạng phức tạp hơn.

Gemini Notebook

Trường tiếp nhận cục bộ (Local Receptive Fields)

Kiến trúc này đã truyền cảm hứng cho Mạng Nơ-ron Tích chập (CNN). Hình 14-1 cho thấy thay vì kết nối với mọi pixel (như MLP), mỗi nơ-ron lớp tích chập thứ nhất chỉ kết nối với một vùng hình chữ nhật nhỏ của hình ảnh đầu vào. Nơ-ron ở lớp thứ hai lại kết nối với một vùng nhỏ của lớp thứ nhất, cứ như vậy tạo ra mạng phân cấp.

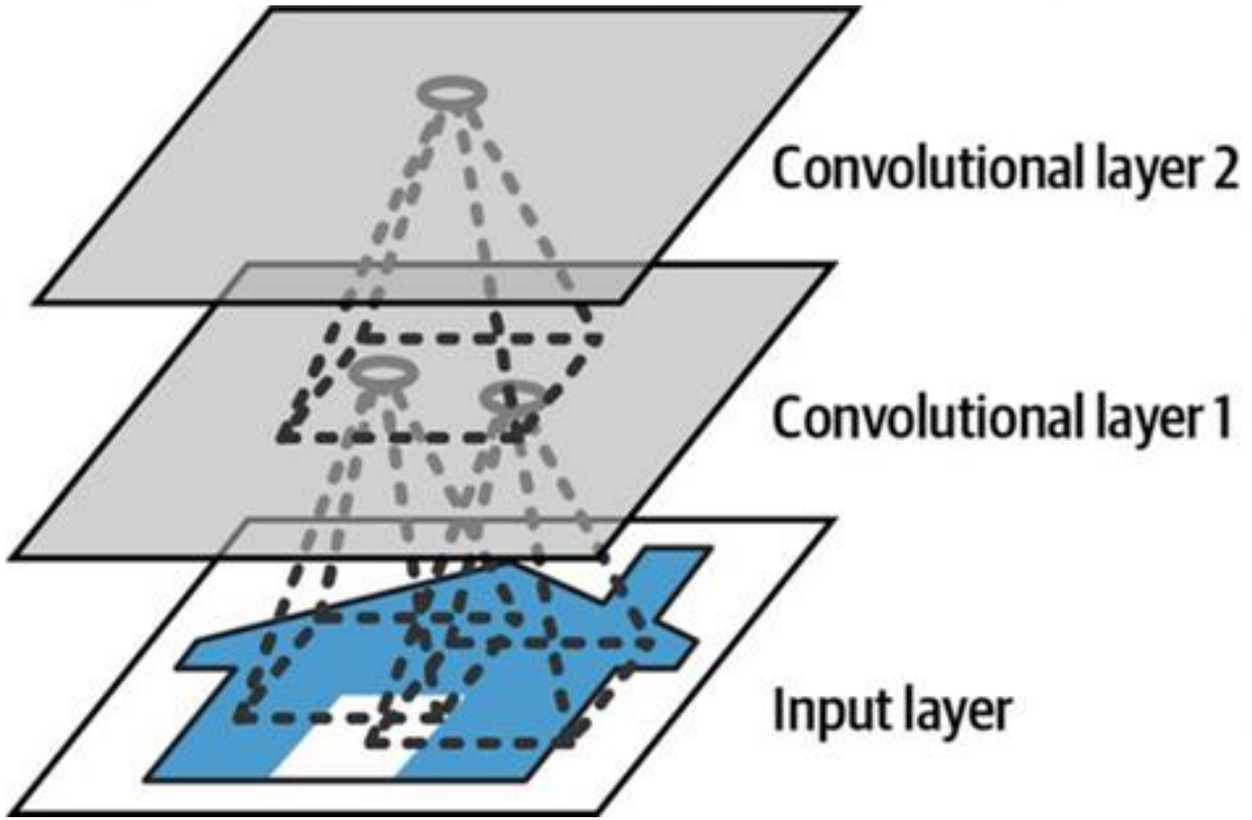


Hình 14-1: Kiến trúc phân cấp của CNN

Gemini Notebook

Lớp Tích chập (Convolutional Layer)

Lớp Tích chập là khối xây dựng cốt lõi của CNN. Một nơ-ron ở vị trí (i, j) của một lớp nhất định được kết nối với đầu ra của các nơ-ron ở lớp trước trong vùng từ hàng i đến $i + f_h - 1$, cột j đến $j + f_w - 1$.
Hình 14-2 minh họa sự kết nối giữa các lớp.



Hình 14-2: Kết nối cục bộ giữa 2 lớp CNN

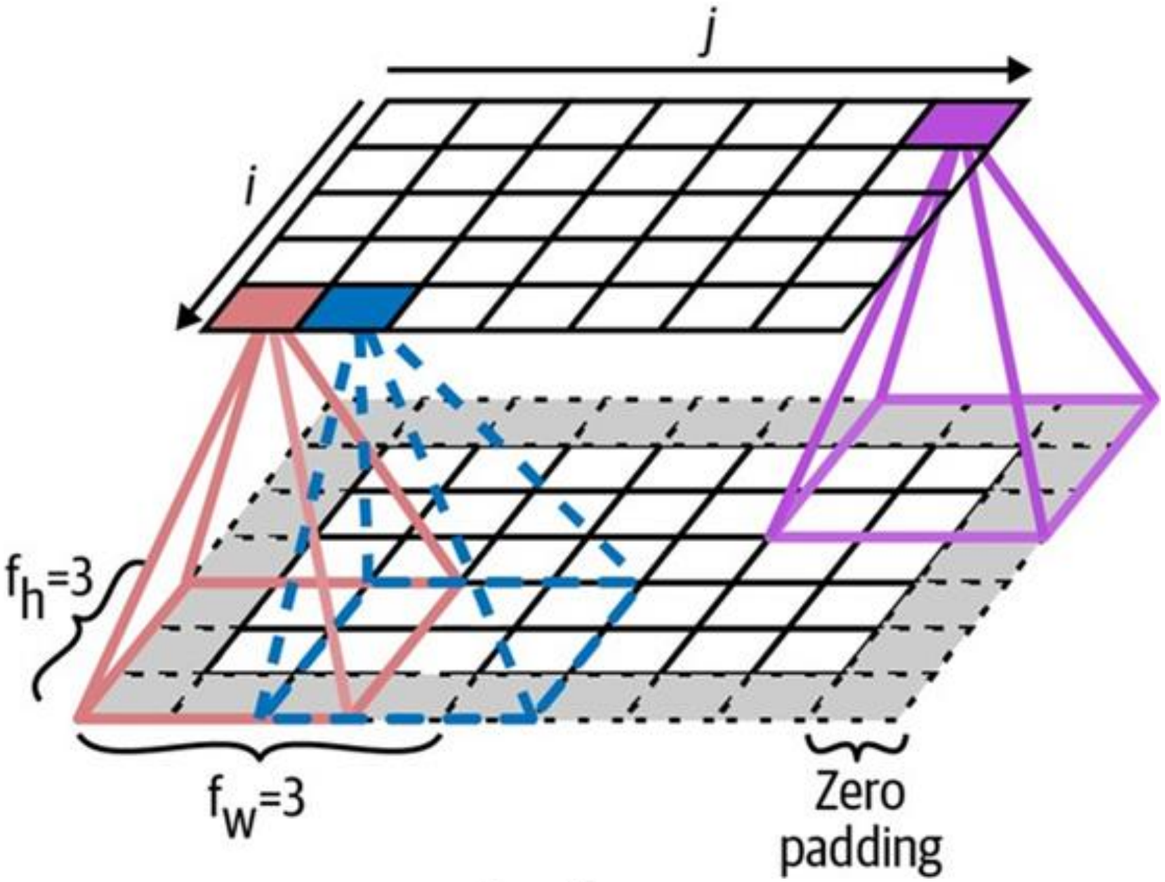
Gemini Notebook

Đệm (Padding)



Nếu tiếp tục áp dụng tích chập, ảnh sẽ nhỏ dần đi sau mỗi lớp. Để giữ nguyên kích thước, ta dùng kĩ thuật **Zero Padding** (đệm thêm các số 0 xung quanh ảnh).

Hình 14-3 (trái): Hình đầu vào 5x5. Hình 14-3 (phải): Hình 5x5 được thêm đệm p = 1 thành 7x7. Tích chập bằng kernel 3× 3 sẽ trả về đúng ma trận đầu ra 5× 5.



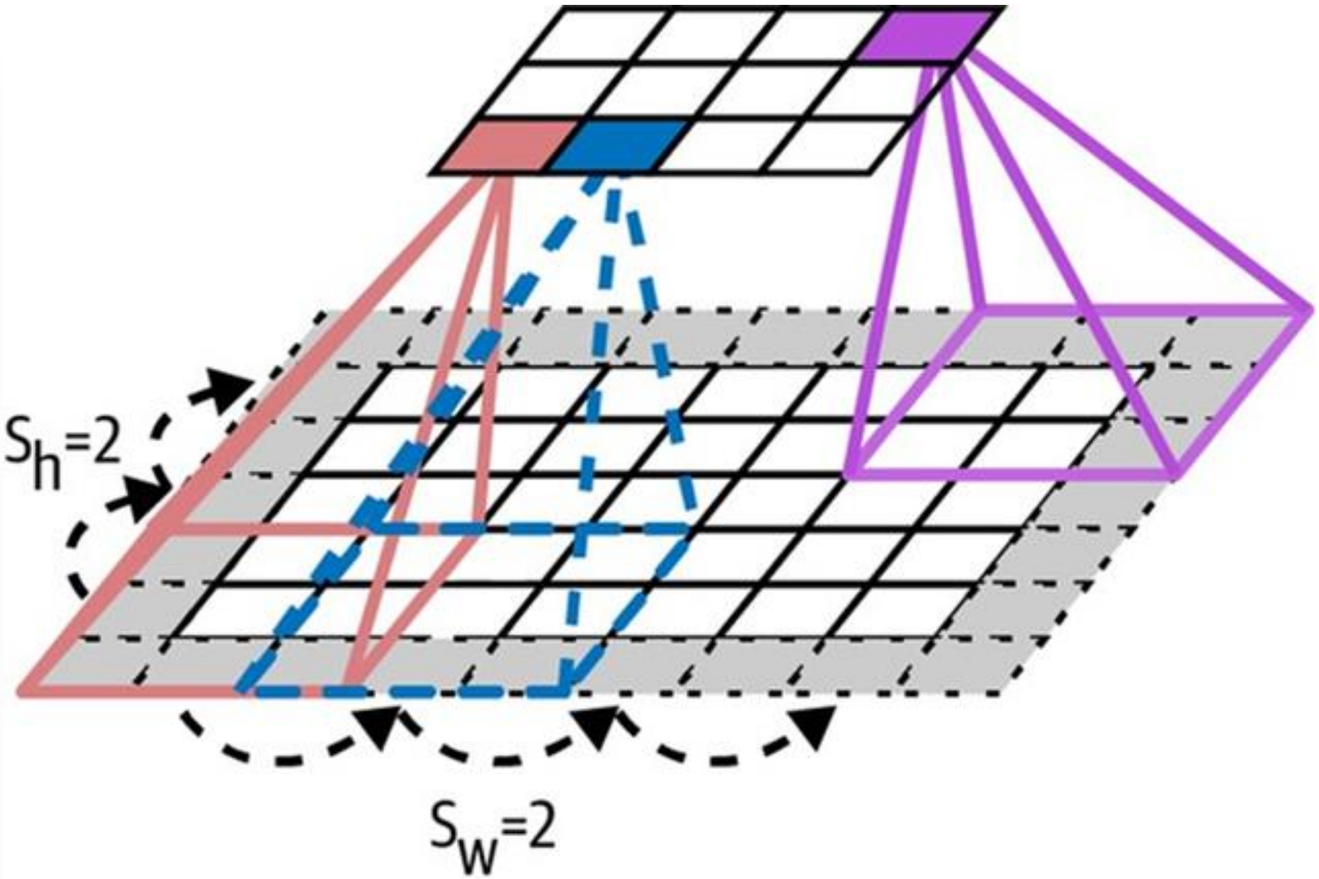
Hình 14-3: Kết nối với Padding

Gemini Notebook

Bước nhảy (Strides)



Nếu muốn giảm kích thước ảnh đầu ra một cách nhanh chóng, chúng ta sử dụng Stride (Bước nhảy) > 1.
Hình 14-4 cho thấy trường tiếp nhận di chuyển 2 ô mỗi lần (s = 2).
Kết quả: Một hình ảnh gốc 5x5 áp dụng kernel 3x3 với padding=1 và stride=2 sẽ cho ra bản đồ đặc trưng có kích thước chỉ còn 3× 3.



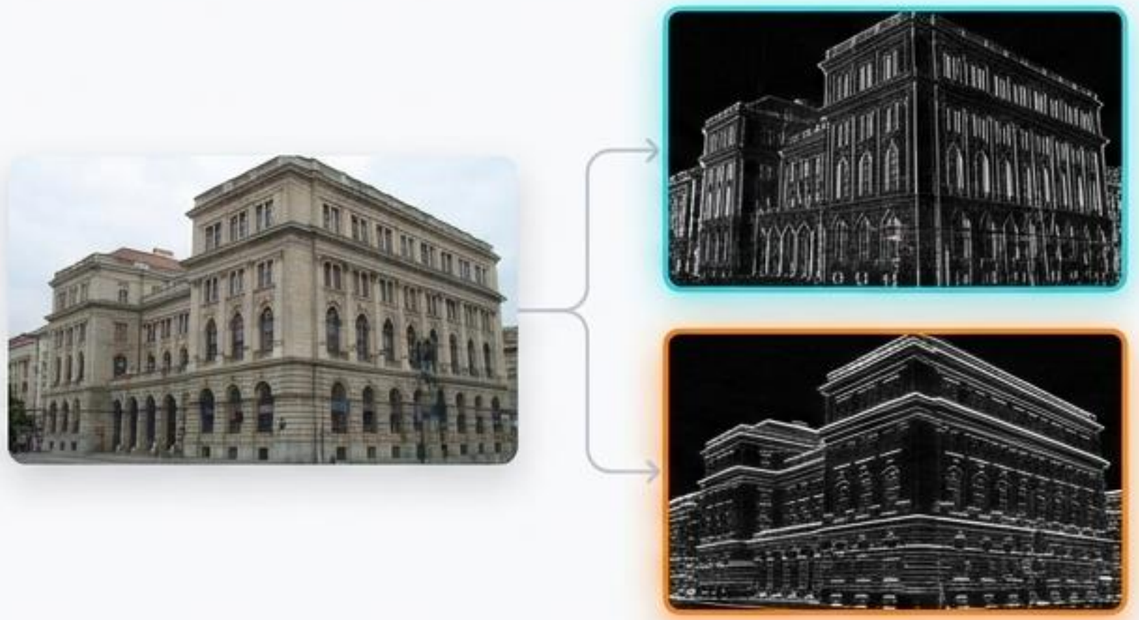
Hình 14-4: Lớp tích chập với bước nhảy Stride=2

Gemini Notebook

Bộ lọc (Filters / Kernels)

Bộ lọc dọc (Vertical filter): Nhấn mạnh các đường nét thẳng đứng.

Bộ lọc ngang (Horizontal filter): Nhấn mạnh các đường nét nằm ngang.



Hình 14-5: Áp dụng các bộ lọc khác nhau lên ảnh

Trọng số của một nơ-ron có thể biểu diễn dưới dạng ma trận kích thước nhỏ, gọi là Filter. Hình 14-5 minh họa hai bộ lọc cổ điển:

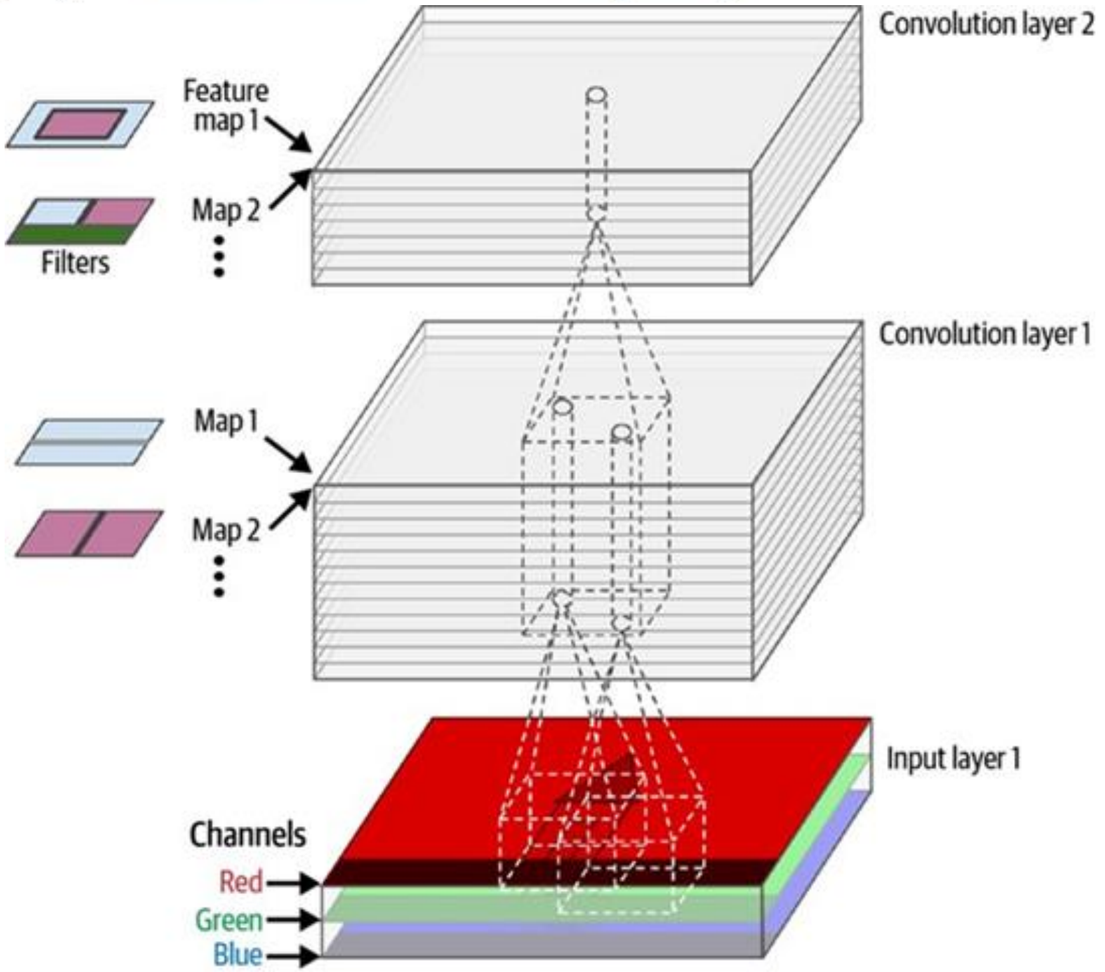
Trong CNN, ta **không tự định nghĩa filter**, mạng nơ-ron sẽ tự học các giá trị bộ lọc tốt nhất cho tác vụ.

Gemini Notebook

Nhiều Bản đồ Đặc trưng (Feature Maps)

Trong thực tế, một lớp tích chập sẽ có nhiều bộ lọc (ví dụ: 64 bộ lọc). Mỗi bộ lọc sinh ra một Bản đồ đặc trưng (Feature map) 2D.

Hình 14-6 mô phỏng quá trình đầu vào 3 kênh (RGB) quét qua nhiều bộ lọc để sinh ra một khối dữ liệu 3D đầu ra (Số kênh bằng số bộ lọc).



Hình 14-6: Đầu vào đa kênh và Đầu ra đa kênh

Gemini Notebook

Padding: ”Valid” vs ”Same”

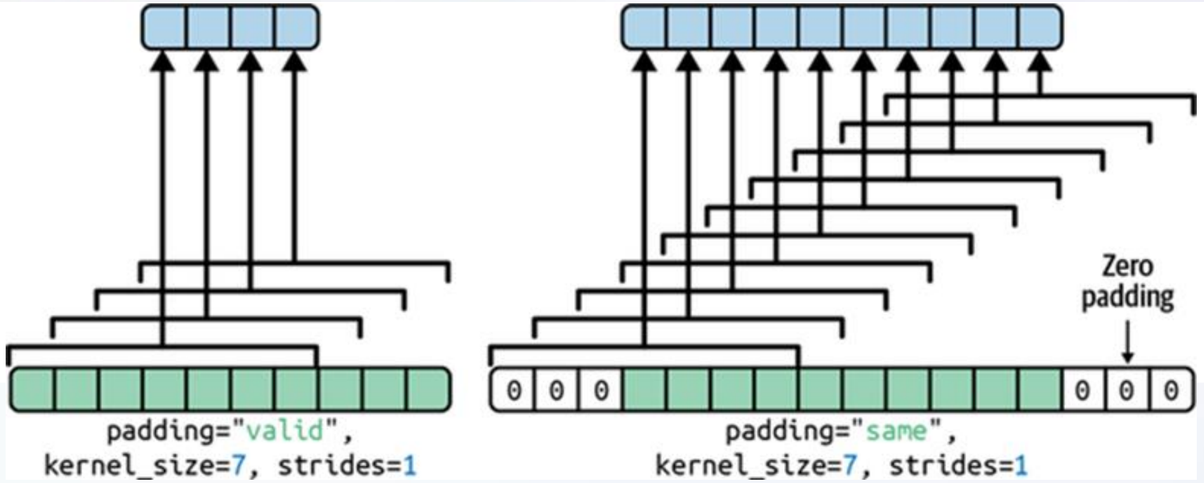
Keras hỗ trợ 2 tham số phổ biến:

padding=“same”

Có padding. Đảm bảo kích thước ảnh ra bằng ảnh vào nếu stride=1 (bên trái Hình 14-7).

padding=“valid”

KHÔNG dùng padding. Bộ lọc có thể vượt ra ngoài biên, làm ảnh bị thu nhỏ lại nhanh chóng (bên phải Hình 14-7).



Hình 14-7: Cài đặt Padding SAME và VALID

Code: Xây dựng Lớp Conv2D trong Keras

Để thêm một lớp Tích chập (Convolution) vào mạng:

```
from tensorflow import keras

model = keras.models.Sequential([
    keras.layers.Conv2D(filters=64, kernel_size=7, activation="relu",
                        padding="same", input_shape=[28, 28, 1])
])
```



filters=64: Mạng sẽ tự học 64 bộ lọc khác nhau.



kernel_size=7: Kích thước bộ lọc là 7× 7. Lớp này đòi hỏi rất nhiều phép tính bộ nhớ, nhưng số lượng tham số (weights) lại rất ít (do có cơ chế chia sẻ trọng số).

Gemini Notebook

Công thức Tính Kích Thước Đầu Ra (Output Dimension)

Kích thước của Bản đồ đặc trưng đầu ra (Feature Map) sau một lớp tích chập được tính bằng công thức:

O: Kích thước không gian đầu ra (Output dimension).

I: Kích thước không gian đầu vào (Input dimension).

$$O = \left\lfloor \frac{I - F + 2P}{S} \right\rfloor + 1$$

F: Kích thước trường tiếp nhận của nơ-ron (Filter size/Receptive field).

P: Lượng đệm (Zero Padding) được thêm vào mỗi phía.

S: Bước nhảy (Stride).

Nếu padding="same", kích thước đầu ra $O = \lceil I/S \rceil$.

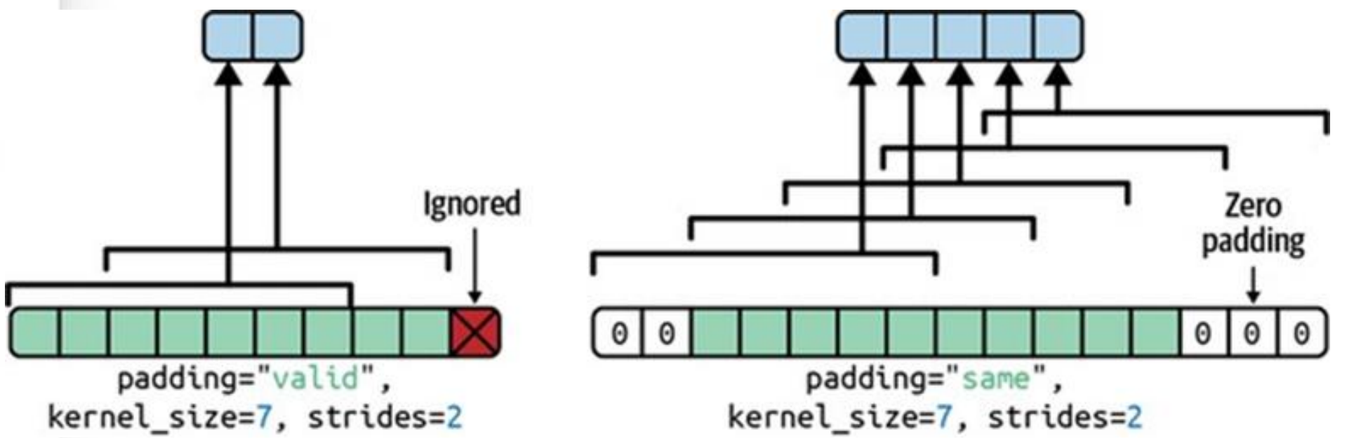
Gemini Notebook

Lớp Gộp (Pooling Layer)

Sau vài lớp Tích chập, số kênh tăng rất nhiều (vd: 256, 512). Cần phải giảm không gian (chiều rộng và chiều cao) để tiết kiệm tài nguyên.

Lớp Gộp (Pooling Layer) ra đời để làm giảm khối lượng dữ liệu (Downsampling).

Hình ảnh 14-8 (nếu có) thường mô tả ý tưởng chung của Downsampling.



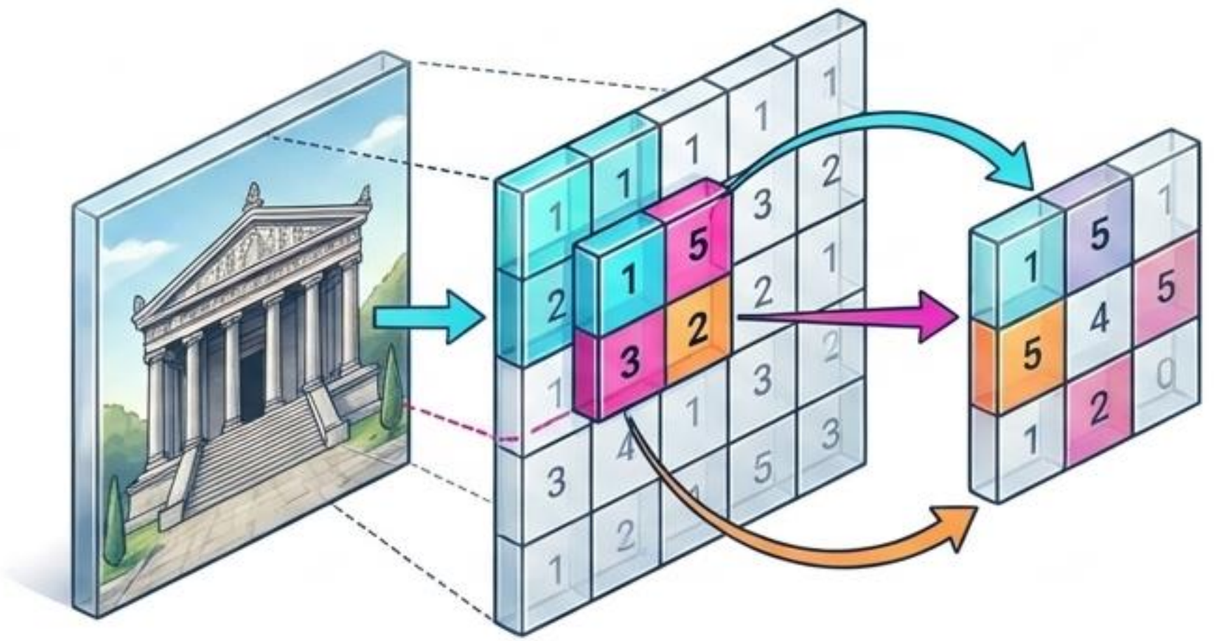
Hình 14-8: Ý tưởng của Pooling

Gemini Notebook

Max Pooling (Gộp Cực Đại)

Phổ biến nhất là Max Pooling. Hình 14-9 cho thấy kernel 2×2, stride=2. Quét qua vùng 2×2, nó chọn ra phần tử có giá trị lớn nhất (ví dụ: max của **[1, 5, 3, 2]** là **5**).

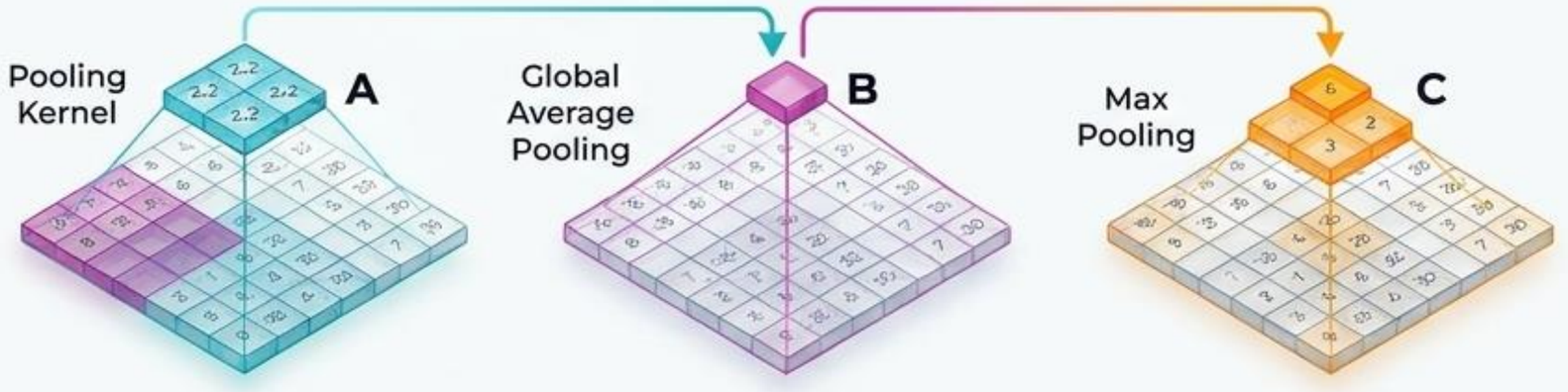
Lợi ích: Chọn ra các đặc trưng nổi bật nhất, và mang lại tính chất bất biến đối với dịch chuyển nhỏ (Translation Invariance).



Hình 14-9: Lớp gộp cực đại (Max Pooling Layer)

Gemini Notebook

Lớp Gộp Trung Bình & Gộp Toàn Cục

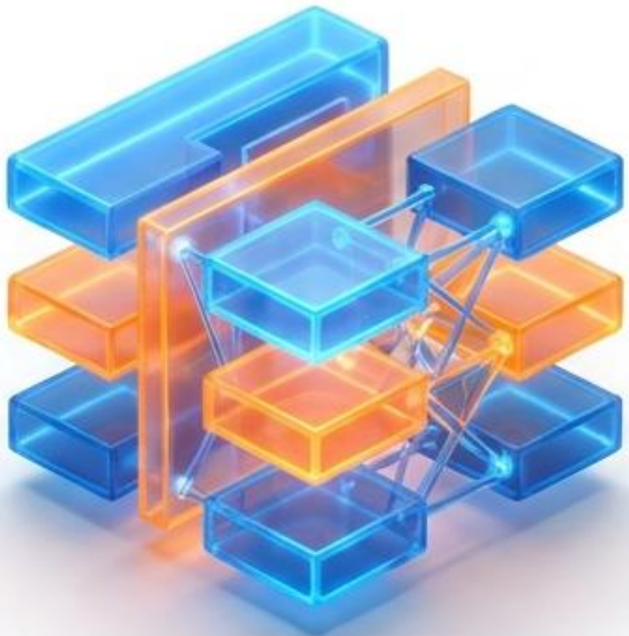


Hình 14-10: Các phương pháp Pooling khác

Average Pooling (Gộp Trung bình):
Thay vì lấy giá trị max, nó tính trung bình tất cả các giá trị trong trường tiếp nhận. Nó từng phổ biến ở LeNet-5, nhưng nay ít dùng hơn Max Pooling vì Max Pooling bảo toàn đường nét mạnh mẽ hơn.

Global Average Pooling (GAP):
Lấy trung bình toàn bộ một Bản đồ đặc trưng (kích thước n x n thu gọn thành 1 con số duy nhất). Rất hay dùng ở lớp cuối cùng trước Softmax, giúp giảm lượng tham số khổng lồ (Hình 14-10).

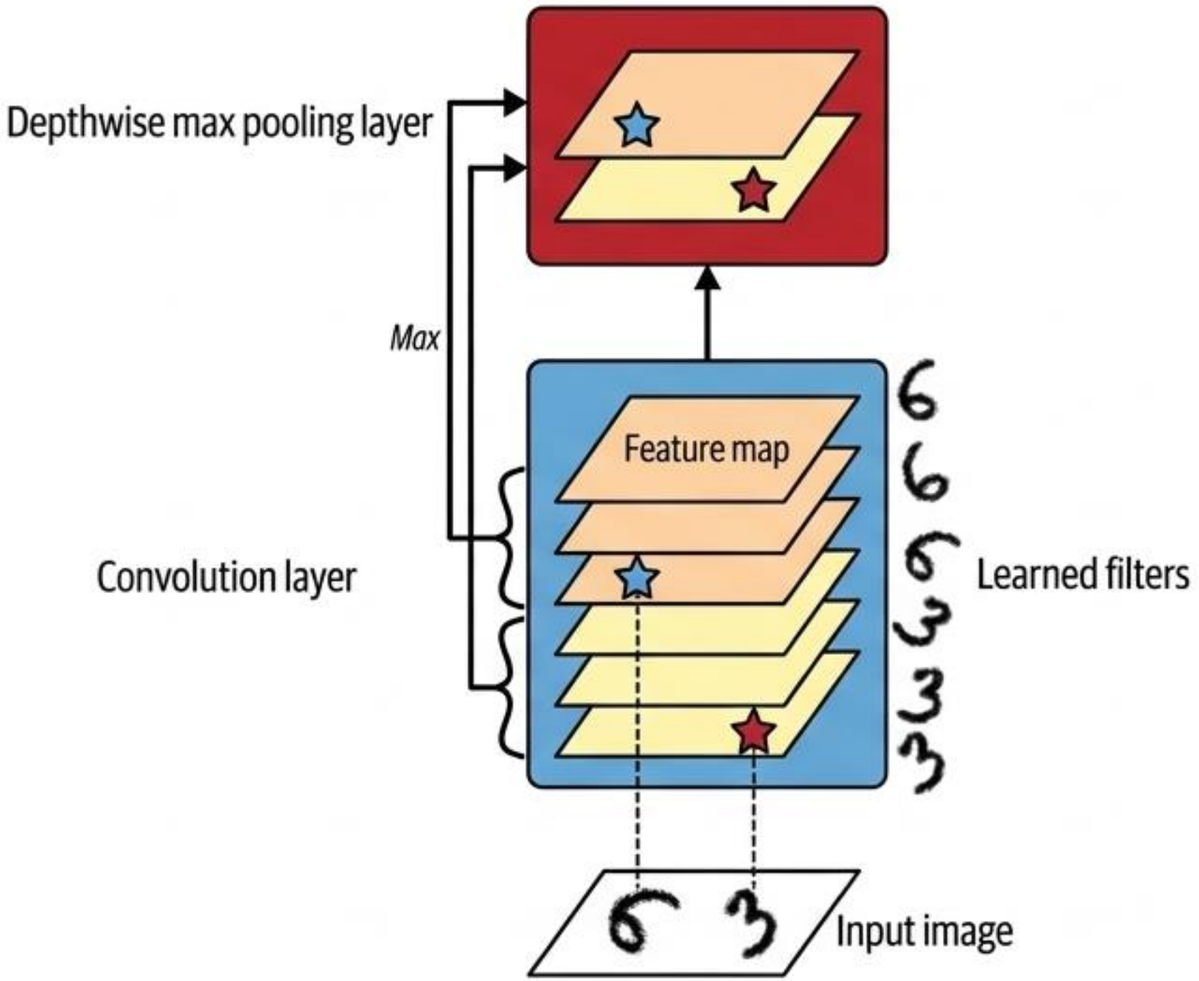
Gemini Notebook



Kiến trúc Mạng Nơ-ron Tích chập (CNN)

Sự phát triển và Các mô hình Kinh điển

Gemini Notebook



Hình 14-11: Tính bất biến nhờ Max Pooling

Tính Bất Biến đối với Dịch Chuyển (Translation Invariance)

Hình 14-11 minh họa: Chữ số C dịch sang phải vài pixel.

Ở các lớp dưới, Max Pooling bảo toàn đường nét chính dù có xê dịch.

Kết quả là vector đặc trưng đầu ra của lớp cuối cùng gần như không đổi.

Điều này vô cùng có lợi cho phân loại, không cần phải dạy mạng học từng vị trí của con mèo trong ảnh.

Gemini Notebook

Tổng hợp Kiến Trúc CNN Tiêu Chuẩn

Hình 14-12 tổng hợp một quy trình chuẩn:



1	2	3	4	5
Cụm (Conv + ReLU + Conv + ReLU + MaxPool). Lặp lại vài lần.	Ảnh nhỏ dần nhưng Dày lên (nhiều feature maps).	Lớp Flatten: Duỗi thẳng thành vector 1D.	Các lớp Dense (Fully Connected) đóng vai trò bộ phân loại.	Output layer (Softmax).

Hình 14-12: Mạng Nơ-ron Tích chập điển hình

Gemini Notebook

Nền móng - LeNet-5 (1998)

- Tác giả: Yann LeCun.
- Ứng dụng: Nhận diện mã bưu điện viết tay trên phong bì.
- Kiến trúc kinh điển đầu tiên kết hợp Convolution và Average Pooling.
- Đặt nền móng cho cuộc cách mạng Deep Learning.



Gemini Notebook

Đột phá - AlexNet (2012)



Chiến thắng vang dội tại cuộc thi **ImageNet ILSVRC 2012.**



Kiến trúc sâu hơn **LeNet-5.**



Lần đầu tiên sử dụng hàm kích hoạt **ReLU**, giúp mạng học cực nhanh.

Lần đầu tiên sử dụng kĩ thuật chống quá khớp đột phá là **Dropout.**



Chạy trên **2 GPU NVIDIA** vì GPU ngày đó không đủ **RAM** để chứa cả mô hình.

Gemini Notebook

Tăng cường Dữ liệu (Data Augmentation)

AlexNet nổi tiếng với việc tạo ra dữ liệu giả khổng lồ từ dữ liệu thật. Hình 14-13 minh họa quá trình:



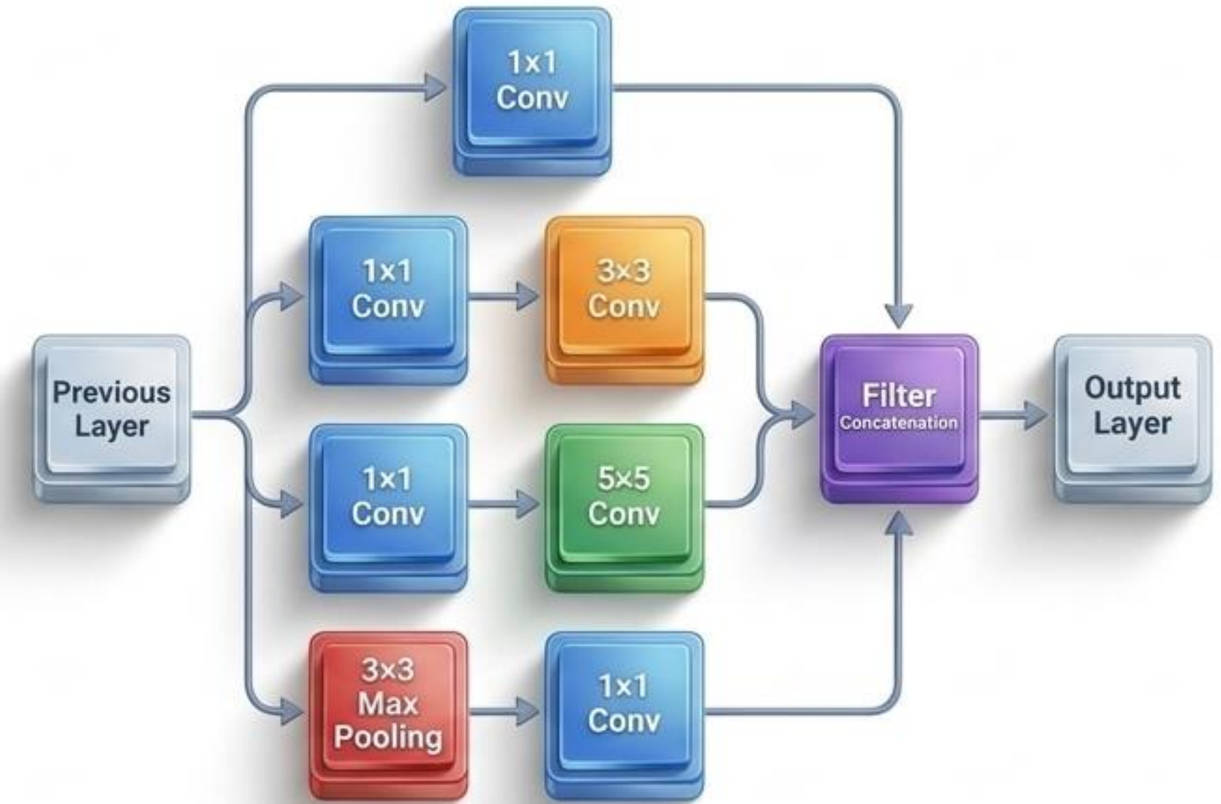
Hình 14-13: Tăng cường dữ liệu (Data Augmentation)

Từ 1 ảnh gốc con ếch, tạo ra hàng chục ảnh đã bị lật ngang, lật dọc, dịch chuyển, xoay, thay đổi màu sắc. Việc này giúp mô hình chống overfit và ép nó học các tính chất bất biến phức tạp hơn.

Gemini Notebook

Mạng GoogLeNet (2014) và Mô đun Inception

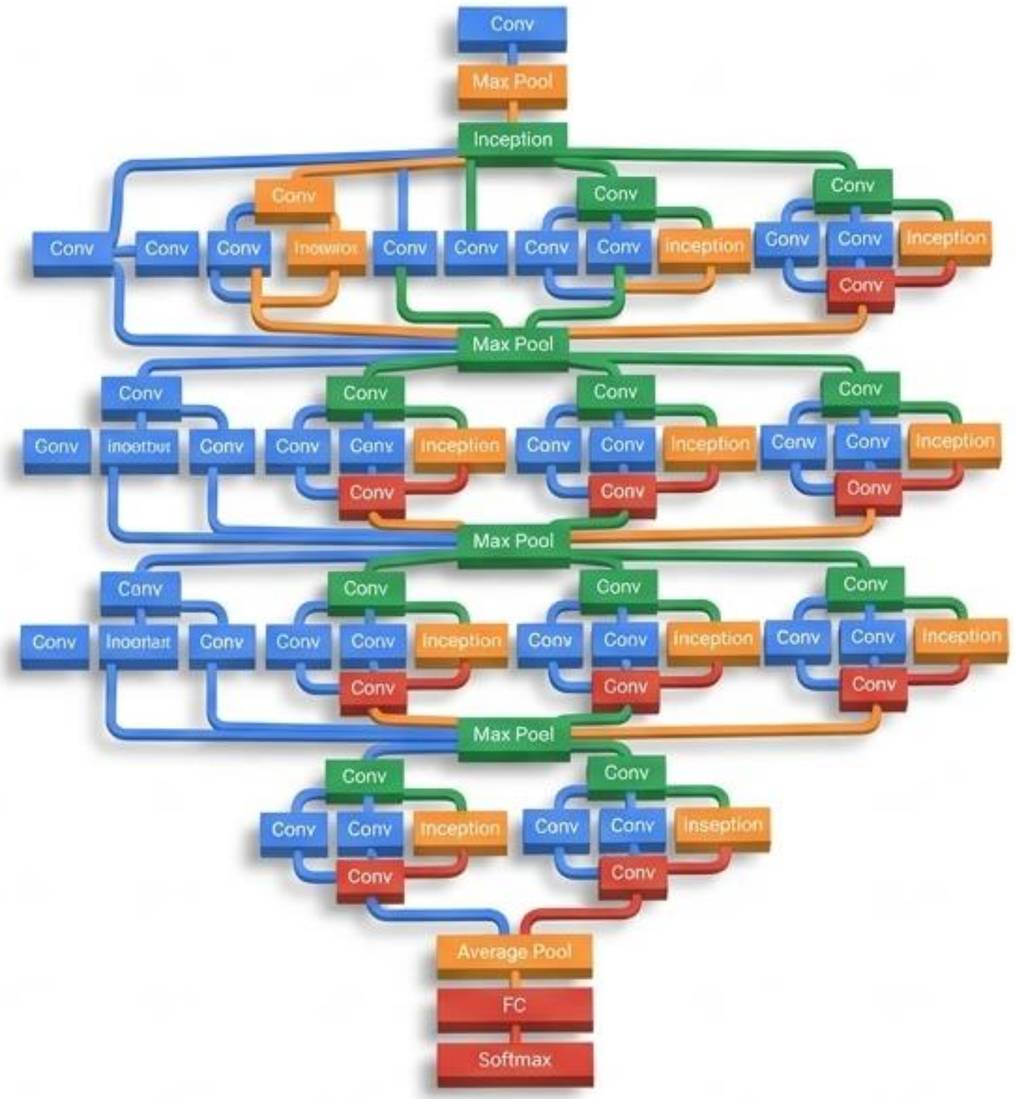
- Lỗi **10%** trên ImageNet xuống còn **<7%**.
- Bí quyết là các mô-đun **Khối Inception** (Hình 14-14).
- Thay vì hỏi: "Nên dùng Kernel 1x1, 3x3 hay 5x5?", GoogLeNet dùng TẤT CẢ cùng lúc trong 1 mô-đun.
- Đặc biệt sử dụng nhiều **Conv 1x1** để "bóp" số lượng kênh, giảm khối lượng tính toán.



Hình 14-14: Kiến trúc một mô-đun Inception

Gemini Notebook

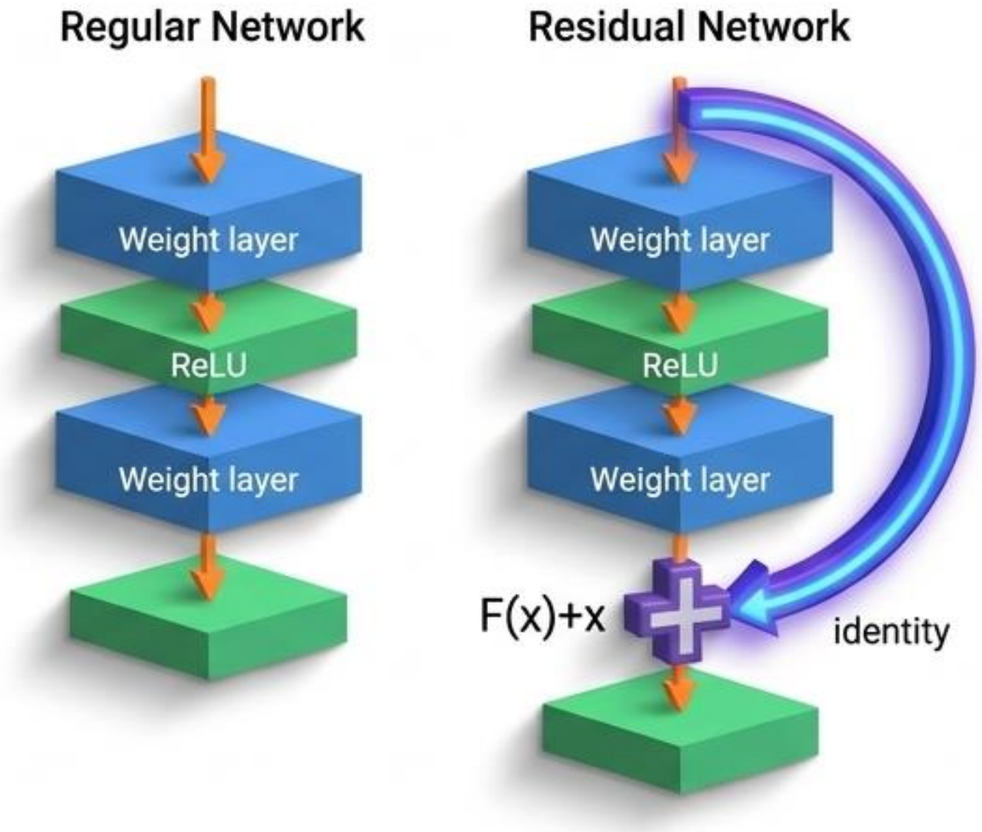
Mạng GoogLeNet Hoàn Chỉnh


Conv
Max Pool
Inception
Conv
Conv
Conv

ResNet (2015) - Residual Networks



- Khi mạng quá sâu (hơn 30 **lớp**), lỗi huấn luyện lại tăng do Gradient suy thoái.
- ResNet giải quyết bằng Kết nối tắt (Skip Connections).
- Thay vì học trực tiếp một hàm $H(x)$, ResNet học phần dư $F(x) = H(x) - x$.
- Hình 14-16 minh họa khối dư (Residual block).



Hình 14-16: Khối Residual Learning cơ bản

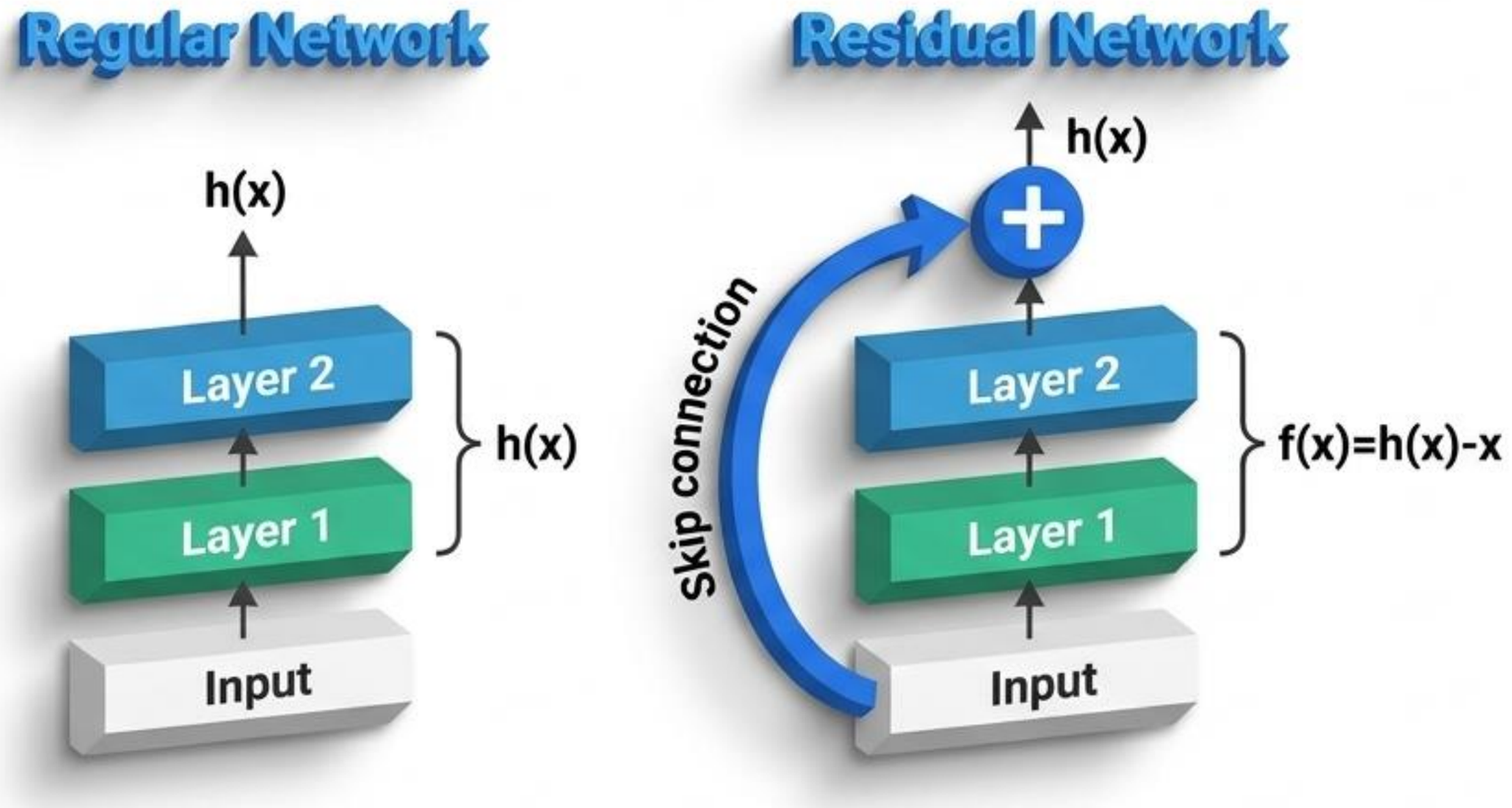
Gemini Notebook

Cơ chế Hoạt động của Skip Connection

Hình 14-17 so sánh **Mạng thường** (Regular Network) và **Mạng Dư thừa** (Residual Network).

Lợi ích: Dòng chảy gradient truyền trực tiếp qua nét liền x, không bị suy giảm.

Cho phép mạng sâu tới **152 lớp** (**ResNet-152**) mà huấn luyện vẫn hội tụ rất nhanh.

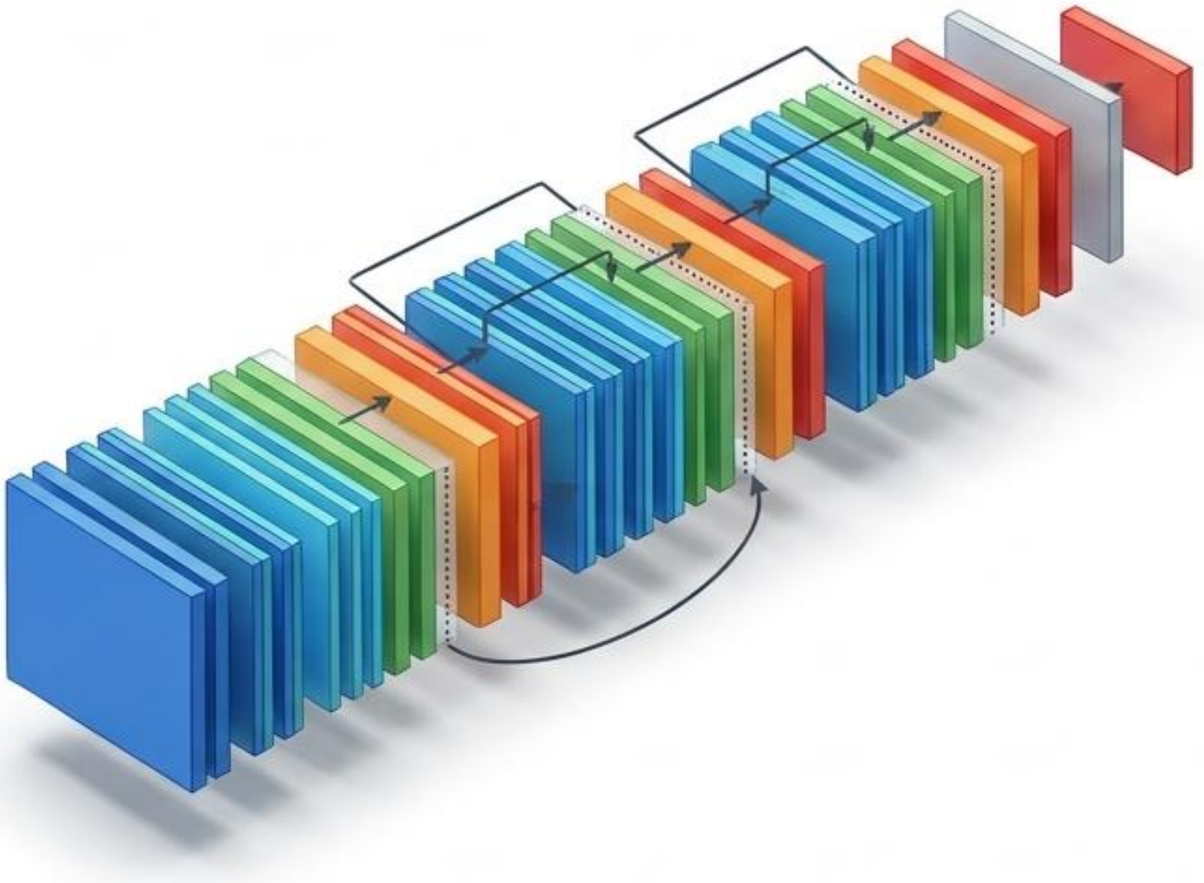


Hình 14-17: Mạng lưới sâu với kết nối bỏ qua

Gemini Notebook

Toàn Cảnh Kiến Trúc ResNet

- **Hình 14-18:** Mạng ResNet xếp chồng hàng chục khối RU liên tục.
- Ở các điểm mốc (đường nét đứt), số kênh nhân đôi, bản đồ đặc trưng thu nhỏ lại một nửa (thông qua Stride=2).



Hình

Gemini Notebook

Code: Xây dựng Khối Residual (ResNet Block)

```python
class ResidualUnit(keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = keras.activations.get(activation)
        self.main_layers = [
            keras.layers.Conv2D(filters, 3, strides=strides, padding="same", use_bias=False),
            keras.layers.BatchNormalization(),
            self.activation,
            keras.layers.Conv2D(filters, 3, strides=1, padding="same", use_bias=False),
            keras.layers.BatchNormalization()]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                keras.layers.Conv2D(filters,1, strides=strides, padding="same", use_bias=False),
                keras.layers.BatchNormalization()]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers: Z = layer(Z)
```

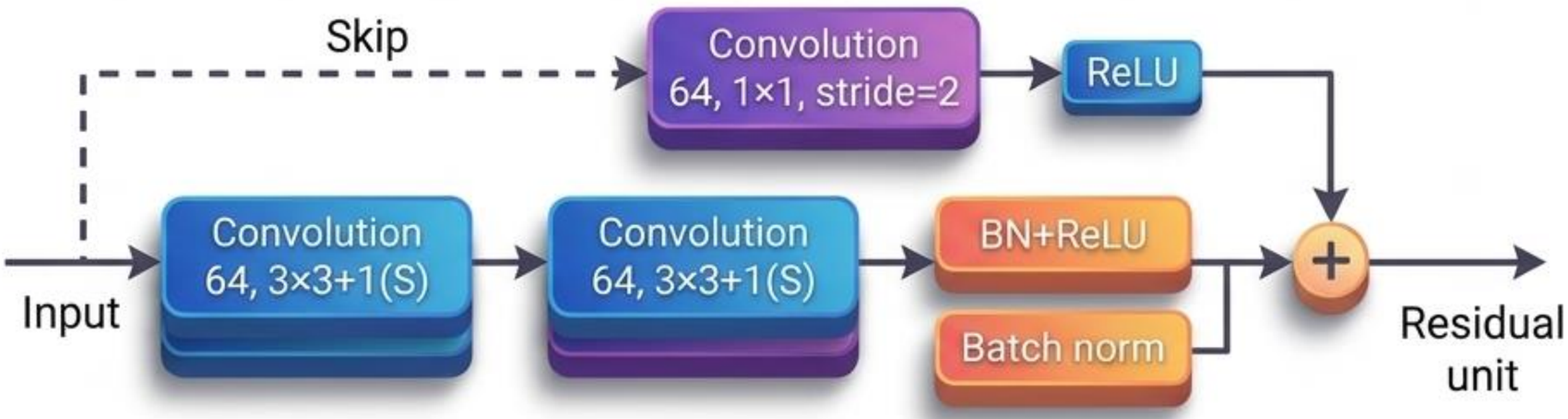
Gemini Notebook

Xử lý Kích thước Kênh qua Skip Connection



- **Khi thay đổi số lượng bản đồ đặc trưng**, ta không thể cộng trực tiếp F(x) + x (Vì khác kích thước ma trận).
- **Giải pháp:** Thêm một lớp Conv 1x1 (stride=2) vào đường chéo (Skip connection) để đồng bộ số lượng bộ lọc.
- Hình 14-19 mô phỏng kĩ thuật này.



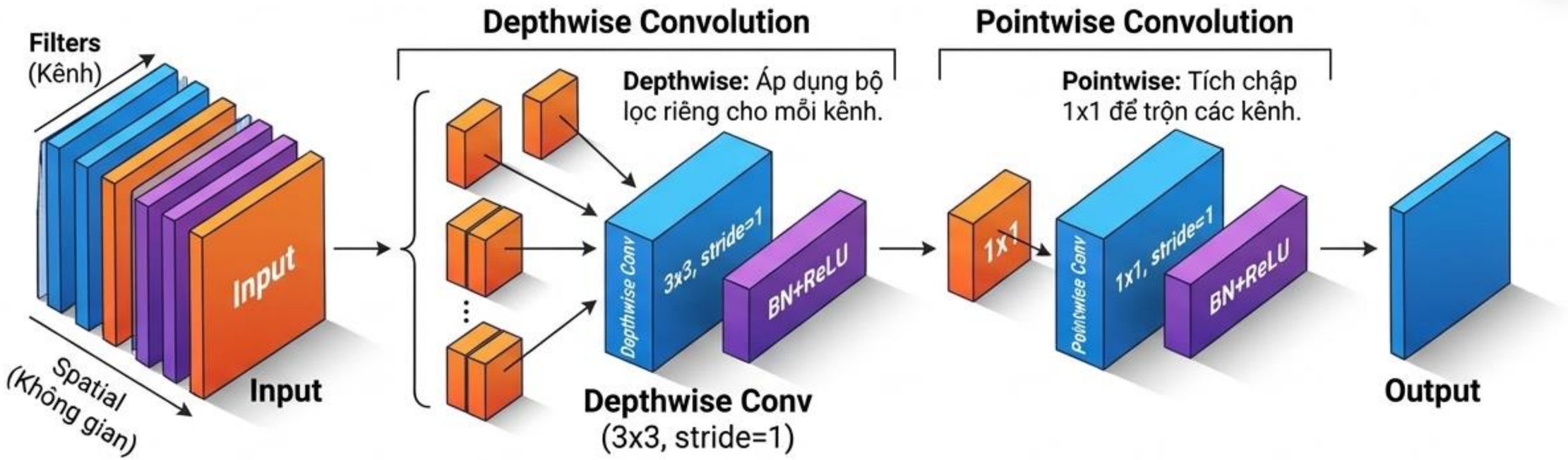


Hình 14-19: *Khối thay đổi kích thước*

Gemini Notebook

Kiến trúc Xception (2016)

- **Tác giả:** Francois Chollet (Cha đẻ của Keras). Tên gọi Xception = ”Extreme Inception”.
- Tách biệt hoàn toàn Không gian (Spatial) và Kênh (Cross-Channel).
- Sử dụng Depthwise Separable Convolution (Hình 14-20) và (Hình 14-21).
- **Ưu điểm:** Tốn ít bộ nhớ và tham số hơn Inception rất nhiều nhưng hiệu năng tốt hơn.



Hình 14-20 & Hình 14-21

Gemini Notebook

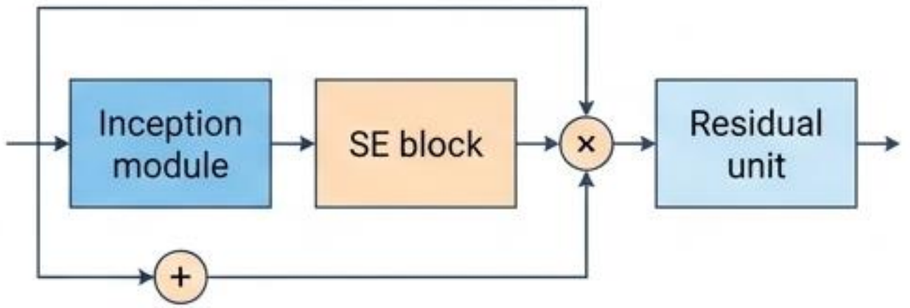
Kiến trúc SENet (2017) - Squeeze-and-Excitation

- Đạt giải nhất ImageNet 2017.

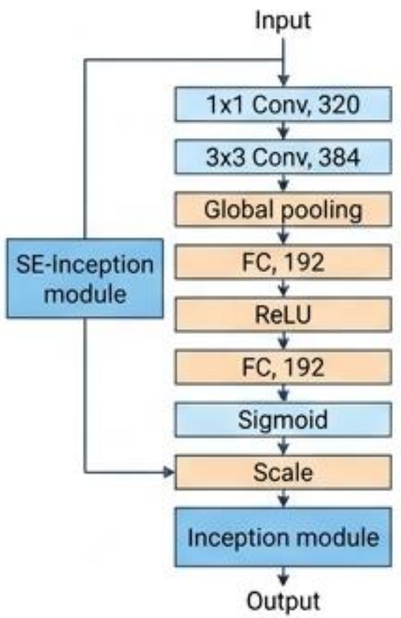
- Nâng cấp mọi mạng hiện có bằng việc gắn thêm Khối SE (Hình 14-22).

- Khối SE tự động học để "đánh trọng số" mức độ quan trọng cho các Bản đồ đặc trưng (kênh). (Vd: Nếu thấy cái mũi, lập tức tăng cường các bộ lọc của con mắt/miệng để tìm khuôn mặt).





Hình 14-22: Khối SE (Squeeze-and-Excitation)



Hình 14-23: Tích hợp SE block vào mô-đun Inception

Gemini Notebook

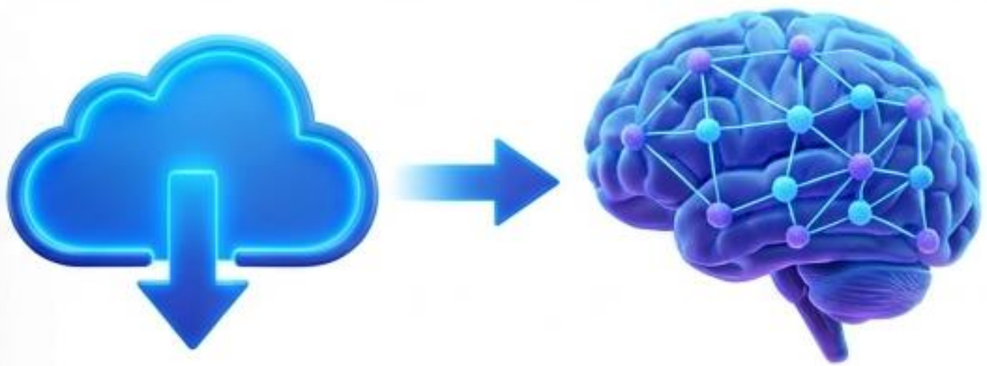
Sử dụng Mô Hình Sẵn Có (Pretrained) trong Keras

```python
from tensorflow.keras.applications import resnet50

# Load model với trong số (weights) da huấn luyện tren ImageNet
model = resnet50.ResNet50(weights="imagenet")

# Xử ly anh về dung kích thuốc yêu cầu
X_resized = tf.image.resize(images, [224, 224])
X_input = resnet50.preprocess_input(X_resized)

Y_proba = model.predict(X_input)
```



Transfer Learning (Học chuyển giao)

Hiếm khi đào tạo mạng ResNet từ đầu (Scratch). Mạng tải xuống đã có khả năng nhận dạng **1000 lớp đồ vật** khác nhau vô cùng chính xác.

Gemini Notebook

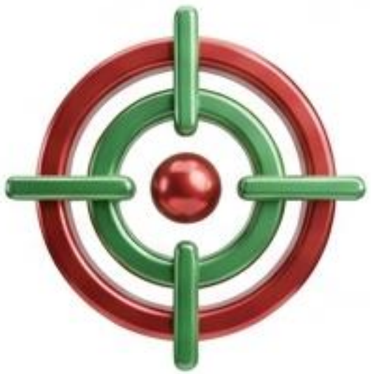
Bài toán Phân loại và Định vị

Phân loại (Classification)



Đó là hoa gì?

Định vị (Localization)



Hoa đó nằm ở đâu trong ảnh?

Đầu ra (Regression): 4 giá trị số thực đại diện cho Bounding Box (Khung chứa)

Tọa độ tâm (x, y) & Kích thước khung $(height, width)$

[**MSE** (Sai số toàn phương cho Bounding Box)] + [**Cross-Entropy** (Cho bộ phân loại)]

Gemini Notebook

Đo lường Khung Chứa: IoU (Intersection over Union)



Hình 14-24: Đánh giá bằng IoU

$$\text{IoU} = \frac{\text{Diện tích Giao}}{\text{Diện tích Hợp}}$$



- ✅ Đánh giá hệ thống dự đoán x, y, w, h.
- ✅ Hộp đỏ và xanh đè lên nhau hoàn hảo ➔ **IoU = 1.0**
- ✅ Lệch hẳn ➔ **IoU = 0**

Gemini Notebook

Thuật toán Non-Max Suppression (NMS)



Hệ thống có thể tạo ra **rất nhiều Hộp Bounding Box** đè lên nhau cho cùng một đối tượng (do **trượt cửa sổ**).

1

Loại bỏ xác suất dưới ngưỡng (vd: < 0.5).

2

Chọn hộp có xác suất **cao nhất**.

3

Tính **IoU** với các hộp còn lại.

4

Loại bỏ (Suppress) các hộp có **IoU ≥ 0.6**.

5

Lặp lại **Bước 2** cho đến khi hết.

Gemini Notebook

Mean Average Precision (mAP)

Đánh giá toàn diện mô hình Object Detection.



mAP

Trung bình AP của
TẤT CẢ các lớp



Đường cong Precision/Recall

Diện tích dưới đường cong = **AP**
(Average Precision) của 1 lớp.



True Positive (Dự đoán Đúng)

(1) Dự đoán đúng Lớp

(2) IoU ≥ ngưỡng (vd: 0.5 hoặc 0.75)

Gemini Notebook

Bài toán Phát Hiện Đối Tượng

Định vị (Localization)

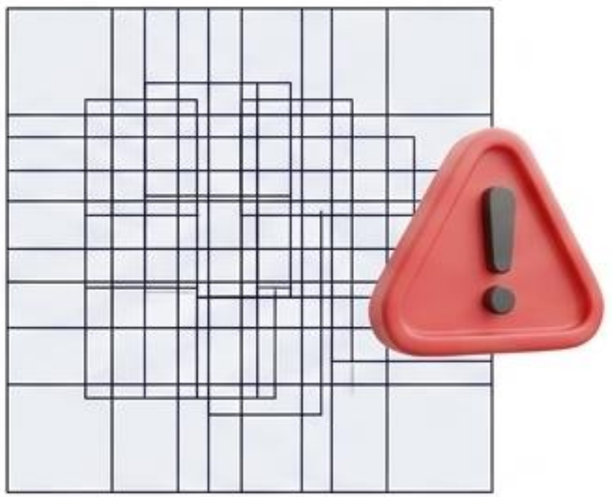
Chỉ tìm **một** đối tượng chính.

Phát hiện (Detection)

Tìm **nhiều** đối tượng thuộc nhiều lớp trong cùng một ảnh.

Cửa sổ trượt (Sliding Window)

(Cách ngây thơ nhất)

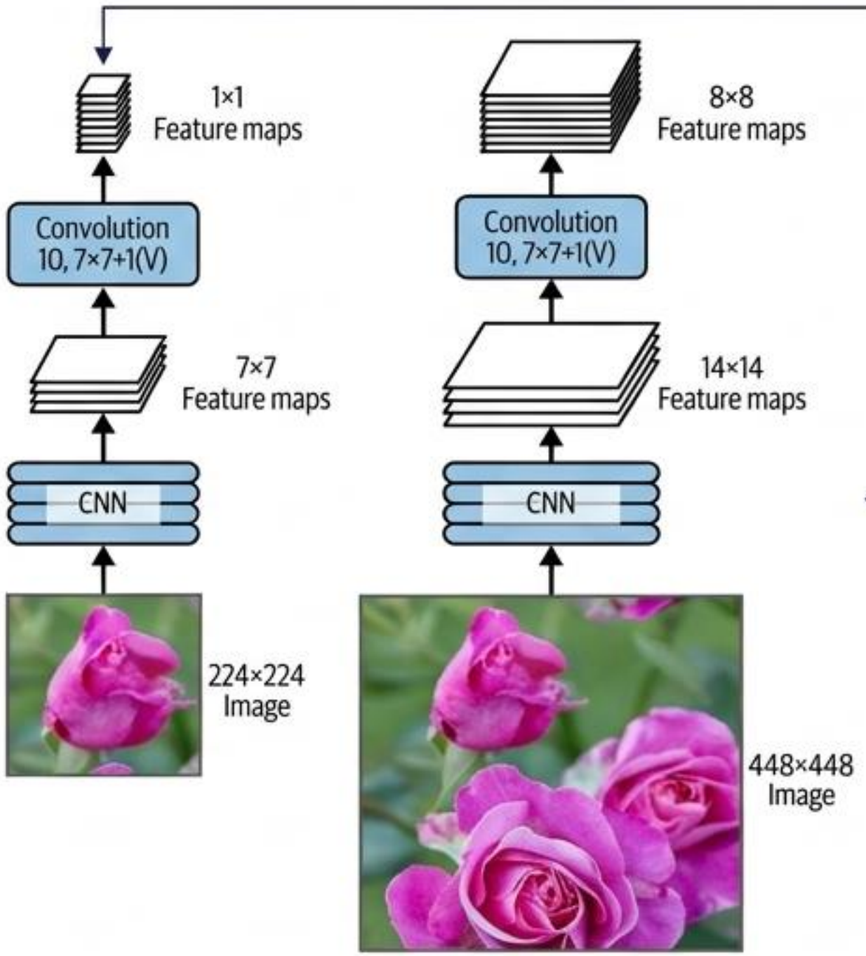


Cắt ảnh thành nhiều hình vuông nhỏ và chạy bộ phân loại trên từng hình.

Rất tốn kém!

Gemini Notebook

Biến CNN thành FCN (Fully Convolutional Network)



Thay lớp Dense ở cuối bằng lớp Tích chập 1×1.

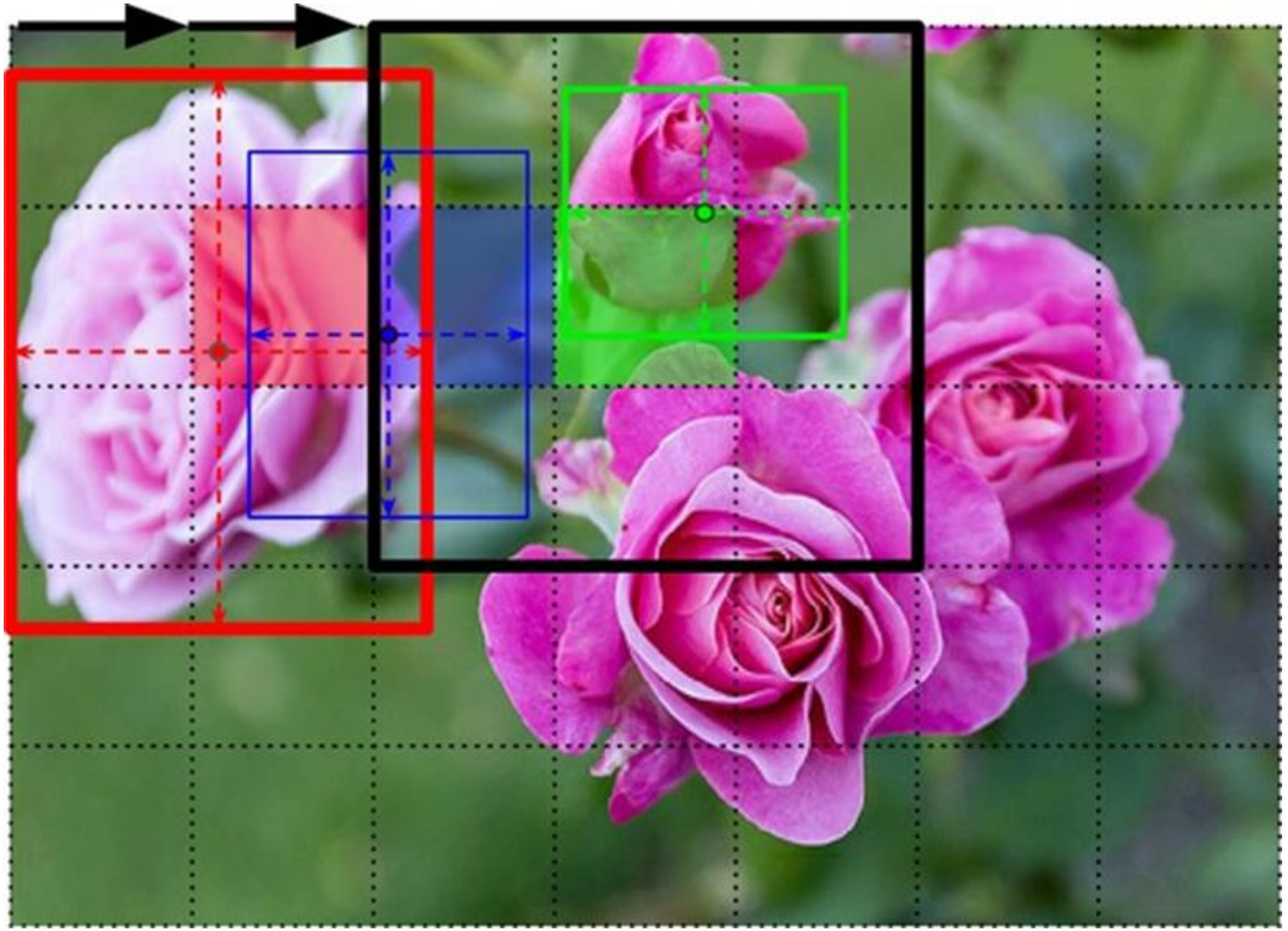
Không có lớp duỗi Flatten ➔ Không ràng buộc về kích thước đầu vào.

Hình 14-26: Chuyển đổi CNN thành FCN

Trở thành mạng tích chập hoàn toàn (FCN)

Gemini Notebook

Minh họa: Lưới dự đoán và Bounding Box



Gemini Notebook

Giải thuật Yolo (You Only Look Once)



Chạy siêu nhanh, thời gian thực (Real-time).

Kiến trúc

Sử dụng FCN tạo ra cuộc cách mạng (2015).

Cơ chế

Chia ảnh thành một lưới (Grid) kích thước 5×7 (tham khảo Hình 14-25).

Dự đoán

Mỗi ô lưới dự đoán Bounding Box cho đối tượng nằm ở tâm của ô lưới đó.

Gemini Notebook

Bài toán Phân Đoạn Ngữ Nghĩa



Không chỉ là khung chứa, hệ thống phải gán **mỗi pixel** một nhãn cụ thể (xe đạp, người, đường phố) **chính xác đến từng đường viền.**

(tham khảo Hình 14-27)

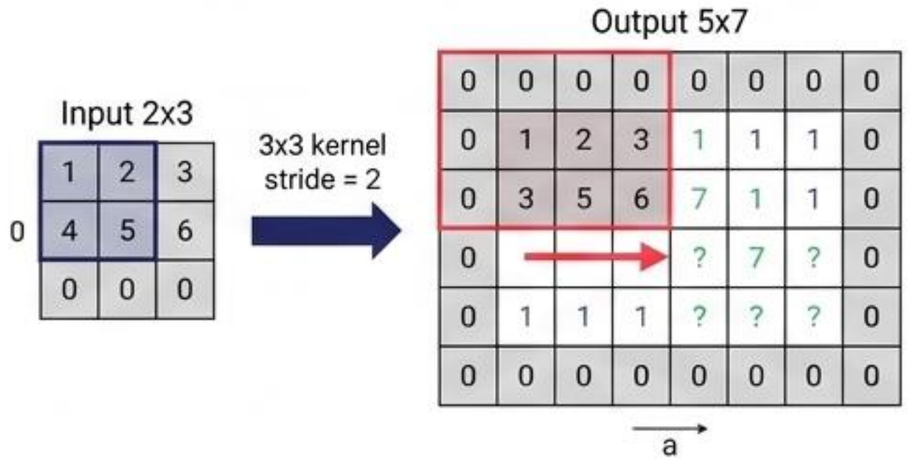


Khó khăn

Qua các lớp Max Pooling, ảnh bị thu nhỏ lại và mất chi tiết viền (Edge details).

Gemini Notebook

Kĩ thuật Tích chập ngược & Tổng Kết Chương 14



Hình 14-28, 14-29: Phép tích chập ngược (Up-sampling)



Tích chập ngược (Transposed Convolution) chèn thêm các số 0 và nội suy phóng to ảnh.

U-Net kết hợp **Skip Connection** lấy chi tiết viền từ các lớp ban đầu.

Cơ bản CNN

Tích chập cục bộ, Chia sẻ trọng số, Pooling.

Lịch sử Kiến trúc

LeNet ➔ AlexNet (ReLU) ➔ GoogLeNet (Inception) ➔ ResNet (Skip) ➔ SENet.

Thị giác Nâng cao

FCN ➔ YOLO (Phát hiện) & U-Net (Phân đoạn).

Gemini Notebook